practice_test_1

SN1-F25 Practice Midterm Questions

The questions below should give you more practise with the kinds of questions you should expect to see on Tuesday's midterm.

Try these problems over the weekend. **See if your solutions work by entering them into Pycharm!**

Solutions will be posted on LEA Monday, Oct 27. Feel free to ask for help before then.

Before you begin

Please read the following notes carefully before beginning the practise midterm questions.

Reference sheet

During the exam, I will provide you with **a reference sheet** printed in your exam booklet. When doing the practise problems, try to only use this sheet when making your solution!

- Reference sheet: click here.

Sample problems from the course webpage

Make sure you have looked over the following practise problems that have been posted already:

- Practise with variables and datatypes
- Practise with operators
- Practise with loops
- Practise with conditional statements
- Practise with custom functions
- Practise with input functions

Material that is NOT covered on the midterm

There are some new sections on the course website that will NOT be on the midterm:

- **Lists**
- **Processing Data**

Anything else on the course website IS something we've covered, and could be on the midterm i.e. everything up to and including **Turtle Graphics**.

Question 1 - Never tell me the odds! 🧮

Part 1

Complete the Python function below so that it **prints** all the odd numbers from 11 to 111 (inclusive, that means include 111!).

```
def never_tell_me_the_odds():
    # your solution goes here
```

```
# This code should go after your function definition. Test that it works in Pyc
never_tell_me_the_odds()
```

Part 2

Change your solution to the above function so that it can take two parameters: `start` and `end`. It should use those parameters to control where the count of odd numbers starts and ends. See below:

```
# Change the function definition so it takes two parameters: start and end
def never_tell_me_the_odds(                                                    ):
    # your solution goes here
```

```
# This code should go after your function definition. Test that it works in Pyc
never_tell_me_the_odds(11, 111) # prints all odd numbers from 11 to 111 inclusi
never_tell_me_the_odds(6, 22)   # prints all odd numbers from 7 to 21 inclusive
```

```
never_tell_me_the_odds(-12, 41) # prints all odd numbers from -11 to 41 inclusi
```

Question 2 - Study Tracker ⏰

Part 1

Your parents are on your case because they think that you are not spending enough time studying. You of course deny this. Since your parents don't believe you, you make an agreement with them where they can track the number of hours that you are home studying at your desk over the course of a week.

Create a Python program that:

- Calculates the total hours studied over a period of 1 week, using this sample data: `2.1, 1.0, 0.74, 3.2, 2.2, 4.0, 0.15`
- Prints the total amount studied, formatted to two decimal places

I've provided you a `for` loop definition you can use for this problem.

```
total_hours_studied = 0
for hours_studied in [2.1, 1.0, 0.74, 3.2, 2.2, 4.0, 0.15]:
  # complete the for loop!
```

```
print(total_hours_studied)
```

Part 2

Modify the loop above so that, in each of the following cases, a different message is printed to the console:

- If the number of hours study is 0
- Else, if the hours of study is less than 10
- Else, if the hours of study is less than 20
- Else, if the hours of study is less than 40
- In all other cases

When trying this at home, change the test data (the list of numbers for the for loop) and convince yourself that your solution works in many cases! (lots of hours, not many hours, etc.)

Question 3 - How Much Things Cost

Part 1

Your friend wants to determine the raw cost of the food they buy at the grocery store. They want you to write a program to help figure this out!

They know the following:

- S: final selling price of the product
- G: grocery store markup factor
- T: transportation distance (in kilometers)

They want you to figure out the following:

- x: the raw cost of the product

They have the following formula you can use:

Final selling price with store markup and transportation:

$$S = 2.3Gx + 0.42T$$

Write a Python function that takes the information your friend knows (`S` , `G` , and `T`) as input and returns the raw cost (`x`) of a given product. See below:

```
# Complete the function below so that it follows all the requirements stated ab
def raw_cost(                                                    ):
    # your function code goes here
```

```
# When you're finished, the following code should work! Test it yourself on Pyc
selling_price = float(input("Enter the selling price: "))
grocer_markup = float(input("Enter the grocer markup factor: "))
```

```
transportation_distance = float(input("Enter the transporation distance: "))
print("The farmer's total costs are: ", raw_cost(selling_price, grocer_markup,
```

Part 2

Modify the function above so that the following conditions are obeyed:

$$S = 2.3Gx + \alpha T$$

Where α is:

- 0.42 when T is less than 100
- 0.84 when T is greater than 100, less than 250
- 1.23 when T is greater thatn 500

Question 4 - The Burrito Brothers

Your distant relatives makes delicious burritos, and you believe they should start selling them. They've come up with the following pricing model:

- The first two burritos are $12 each
- Any additional burritos are $6 each
- The 11th "bonus" burritos is $1.11
- All following burritos are $11 each

Create a Python function that:

- Has one parameter `n`, representing the number of burritos
- Returns the total cost of the burritos.

This time, I haven't given you any starter code – you should define the function, and show it in use, yourself. See the previous questions for examples!

Question 5 - A Time for Turtles

Part 1

In the space below, use the turtle library to draw a square using a red pen, with corners at each of the following coordinates:

- `(100, 100)`
- `(200, 100)`
- `(200, 200)`
- `(100, 200)`

```
import turtle
# your solution goes here









turtle.mainloop()
```

NOTE: Move the turtle so that there are NO lines EXCEPT for the square.

> HINT: Don't forget about the **reference sheet** i linked at the beginning of this document!

Part 2

Write a function that will make a Turtle draw a square of size `n`.

Using that function, draw three more squares of size `200`, `300`, and `400`.

All four squares should start with their bottom left corner on the coordinate `(100, 100)`

I've provided some helpful code you can use

```
# Note: you don't have to import turtle again
```

```
def turtle_square(n):
    # Finish this function!
```

```
# ... rest of your code goes here ...#

turtle_square(200) # draws a square of size 200 at the turtle's current positio

# Note: don't worry about writing turtle.mainloop()
```

Part 3

Make a new function `turtle_centered_square(n)`.

This function should draw a square that is **centered on the turtle's current position**.

That is, if the turtle is at the position `(0, 0)`, a square with size `n=100` should have it's four corners on `(-50, -50), (50, -50), (50, 50), (-50, 50)`.

Then, use that function to draw three centered squares of size `80`, `40`, and `20` at the coordinates `(-160, -160)`.

```
# Note: you don't have to import turtle again
```

```
# Note: don't worry about writing turtle.mainloop()
```

Looking for more practise?

Get started on Assignment 3! It covers the same material as the test, and will be great practise itself.